

Secure Closed-Loop Illumination Control via Additively Homomorphic LWE Encryption on Edge Hardware

Mir Maiti (230102057) Anshuman Mondal (230102009)

Department of Electronics & Electrical Engineering

Indian Institute of Technology Guwahati

1. Introduction & Problem Statement

Cyber-physical systems (CPS) routinely transmit raw sensor telemetry to remote servers for control computations. In sensitive environments, intercepting this telemetry enables adversaries to infer operational states via traffic analysis. Furthermore, edge microcontrollers lack the computational capacity for standard heavy-weight homomorphic cryptographic schemes.

This project implements a fully operational homomorphic closed-loop illumination control system. A BH1750 light sensor on an Arduino Mega encrypts lux readings using Learning With Errors (LWE). A remote Python host executes Proportional-Integral-Derivative (PID) arithmetic entirely in the ciphertext domain. The encrypted PWM signal is returned and decrypted on-device to drive an LED array within a 3D-printed enclosure.

2. Motivation & Contributions

- **Zero-trust architecture:** Plaintext sensor values never leave the local node.
- **Resource-efficient HE:** Our LWE scheme uses 8 KB SRAM with sub-18 ms decryption.
- **End-to-end encryption:** Both uplink and downlink channels carry only ciphertexts.
- **Encrypted anti-windup:** Integral clamping is achieved via ciphertext proxy evaluation.
- **Optimized arithmetic:** Modulus $Q = 2^{31} - 1$ enables rapid 32-bit operations on AVR.

3. System Architecture

The hardware testbed consists of an Arduino Mega 2560, a BH1750 I²C light sensor, and a quad-LED array (PWM pins 4–7). These are housed in a custom 3D-printed opaque enclosure to isolate internal illumination. A PC running Python 3 acts as the compute node via 115200 baud USB-serial.

The 300 ms control loop operates as follows:

1. Arduino samples BH1750 and applies a 10-sample moving average filter.
2. Filtered lux is encrypted (LWE) and transmitted.
3. Python host computes homomorphic PID output.
4. Arduino decrypts the PWM ciphertext and drives the

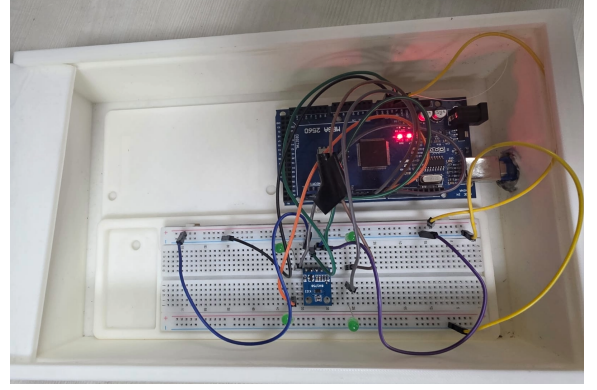


Figure 1: Hardware setup: Arduino Mega, BH1750 sensor, and LED array within the 3D-printed dark box.

LEDs using exponential smoothing.

4. Cryptographic Framework

We implement an asymmetric LWE scheme utilizing hardware-friendly parameters: $N = 4$, $M = 8$, and modulus $Q = 2^{31} - 1 \approx 2.1 \times 10^9$.

The secret key $\mathbf{s} \in \mathbb{Z}_Q^N$ resides strictly in MCU flash memory. The public key $(\mathbf{A}, \mathbf{b})$ consists of matrix $\mathbf{A} \in \mathbb{Z}_Q^{N \times M}$ and vector $\mathbf{b} = \mathbf{A}^T \mathbf{s} + \mathbf{e} \pmod{Q}$. Plaintext x is scaled by $\Delta \approx 2$ and encrypted as:

$$\text{Enc}(x) = (\mathbf{u}, v), \quad v = \sum_{j \in S} b_j + \Delta \cdot x \pmod{Q} \quad (1)$$

Decryption accurately recovers x by evaluating the phase:

$$\hat{x} = \left\lfloor \frac{v - \langle \mathbf{s}, \mathbf{u} \rangle \pmod{Q}}{\Delta} \right\rfloor \quad (2)$$

Algorithm 1 LWE Encryption (Edge Node)

Require: Plaintext x , PK $(\mathbf{A}, \mathbf{b})$, modulus Q , scale Δ

- 1: Select random subset $S \subseteq \{1, \dots, M\}$
 - 2: $\mathbf{u} \leftarrow \sum_{j \in S} \mathbf{A}_j \pmod{Q}$
 - 3: $v \leftarrow \sum_{j \in S} b_j + \Delta \cdot x \pmod{Q}$
 - 4: **return** Ciphertext $(\mathbf{u}, v)$
-

4.1. Homomorphic Operations

LWE supports additive homomorphisms. Given $\text{CT}(x_1)$ and $\text{CT}(x_2)$, their sum is evaluated as:

$$(\mathbf{u}_1 + \mathbf{u}_2, v_1 + v_2) = \text{CT}(x_1 + x_2) \quad (3)$$

Scalar multiplication by a constant k is achieved via:

$$(ku, kv) = \text{CT}(k \cdot x) \quad (4)$$

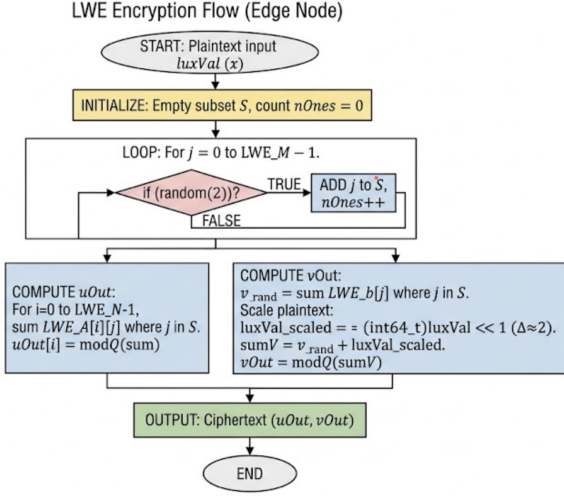


Figure 2: LWE cryptographic flow and homomorphic arithmetic logic.

5. Encrypted PID Control Logic

The Python server maintains an encrypted integral accumulator. It applies specific, scaled control gains ($K_p = 1.2$, $K_i = 0.08$,) in the ciphertext domain.

5.1. Error & Proportional Computation

The target setpoint ($r \approx 110$ lux) is introduced as a plaintext scalar shift. Encrypted error is calculated by negating the lux ciphertext:

$$\text{CT}(e_t) = (-u_t, -v_t + \Delta \cdot r) \quad (5)$$

The proportional ciphertext is then computed via scalar multiplication:

$$\text{CT}(P_t) = K_{p,\text{int}} \cdot \text{CT}(e_t) \quad (6)$$

5.2. Anti-Windup Integral Accumulation

Raw error ciphertexts are accumulated sequentially:

$$\text{CT}(I_{\text{raw}})^{(t)} = \text{CT}(I_{\text{raw}})^{(t-1)} + \text{CT}(e_t) \pmod{Q} \quad (7)$$

To prevent integral windup (clamp = ± 200), the server centers the v -component of the accumulator ($\tilde{v}_I$). If $|\tilde{v}_I|$ exceeds the scaled clamp threshold, addition is suspended. The final integral output is $\text{CT}(I_t) = K_{i,\text{int}} \cdot \text{CT}(I_{\text{raw}})$.

5.3. Output Smoothing

The aggregated control signal $\text{CT}(\text{PWM}) = \text{CT}(P_t) + \text{CT}(I_t)$ is transmitted to the MCU. After decryption, an exponential smoothing filter ($\alpha = 0.3$) is applied locally to prevent actuator jitter:

$$\text{PWM}_t = \alpha \cdot \text{PWM}_{\text{raw}} + (1 - \alpha) \cdot \text{PWM}_{t-1} \quad (8)$$

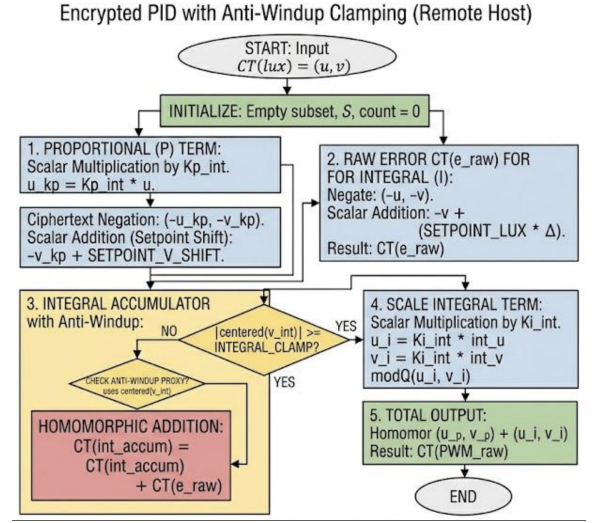


Figure 3: Encrypted PID control loop including proxy-based anti-windup clamping.

6. Implementation & Performance

6.1. Latency & Resource Utilization

Using AVR optimizations for $Q = 2^{31} - 1$, modular reduction requires only bitwise shifts and additions.

- **Encryption Time:** < 5 ms on Arduino Mega.
- **Decryption Time:** ≈ 18 ms.
- **HE Computation:** < 2 ms on Python host.
- **Total Loop Latency:** ≈ 45 ms (fits 300 ms budget).

6.2. Control Results

- **Convergence:** Reached setpoint in ~ 12 cycles.
- **Stability:** Exponential smoothing ($\alpha = 0.3$) completely mitigated high-frequency oscillations.
- **Clamping:** Encrypted integral anti-windup correctly capped accumulation.
- **Noise:** Decryption error bounded to ~ 1 lux unit.

7. Security Guarantees

The architecture ensures the host observes only high-entropy uniform vectors (u_t, v_t) . Without s , lux levels remain indistinguishable. Anti-windup conditional logic operates strictly on ciphertext magnitudes ($\tilde{v}_I$), preventing side-channel leakage of plaintext control metrics.

8. Conclusion

This framework proves the viability of utilizing lightweight LWE for encrypted edge-device automation. By executing proportional-integral arithmetic natively on ciphertexts and applying local output smoothing, the system achieves robust, real-time closed-loop control without compromising sensor confidentiality.